Khulna University of Engineering & Technology
Khulna-9203, Bangladesh
Department of Computer Science and Engineering

{\large \textbf{CSE 3218 - Mobile Computing Laboratory}}

Project Report

Productivitree: A Social Gamified Productivity App for

{\Large \textbf{Smart Personal and Collaborative Productivity}}

Submitted by:

Rysul Aman Nirob ..... Roll No: 2107092
Biposhyan Chakma ..... Roll No: 2107128

Department of Computer Science and Engineering
Khulna University of Engineering and Technology

Submitted to:

[Name of Teacher 1], [Designation]
[Name of Teacher 2], [Designation]

Department of Computer Science and Engineering
Khulna University of Engineering and Technology

Abstract

Productivitree is an iOS productivity application developed using SwiftUI and Firebase to combine task management, habit tracking, focus sessions, study-partner collaboration, gamification, and analytics within a single mobile platform. The system is designed to improve both personal productivity and collaborative accountability by integrating XP rewards, streak tracking, virtual tree growth, real-time study sessions, and weekly performance analytics. Firebase Authentication and Cloud Firestore are used to manage user accounts and cloud data synchronization, while additional service integrations support quote fetching and image upload features. This report presents the project objectives, system overview, architecture, implementation details, major functionalities, work distribution, limitations, and future scope.

1. Objectives

- To develop an iOS productivity platform using SwiftUI and MVVM architecture.
- To implement secure user authentication and cloud persistence using Firebase.
- To support task, habit, and focus-management workflows in one application.
- To enable collaborative study sessions with real-time participant updates.
- To integrate gamification through XP, levels, streaks, and tree growth.
- To provide profile statistics and weekly analytics for user progress tracking.
- To demonstrate a practical mobile solution for improving personal and collaborative productivity.

2. Introduction

Mobile applications have become essential tools for managing everyday productivity. Many users rely on separate applications for tasks, habits, focus timers, and collaboration, which often leads to fragmented workflows and inconsistent progress tracking. Productivitree addresses this issue by integrating these features into a unified iOS application.

The project combines personal productivity management with social accountability and gamification. Users can create and manage tasks, maintain habits, run focus sessions, join study sessions, and view analytics from a single interface. By combining cloud synchronization, responsive SwiftUI interfaces, and progress-based rewards, the application aims to increase motivation, consistency, and engagement.

3. System Overview

Productivitree is a SwiftUI-based mobile application built on the MVVM architectural pattern. It provides several integrated modules including authentication, tasks, habits, focus sessions, study sessions, profile management, gamification, and analytics. The backend relies on Firebase Authentication and Cloud Firestore, while supporting services handle remote quotes, optional image uploads, and tree-world progression logic.

3.1 Core Modules

- Authentication module for registration, login, and session persistence.
- Task management module for CRUD operations, filtering, deadlines, and completion tracking.
- Habit tracking module for daily completion, streak calculation, heatmap visualization, and proof uploads.
- Focus module for timer-based deep work, learning, creating modes, ambient sound support, and Pomodoro flow.
- Study session module for collaborative sessions with create, join, leave, start, end, and delete operations.
- Gamification module for XP rewards, levels, motivational progress feedback, and tree growth stages.
- Profile and analytics module for user statistics, charts, progress bars, and quote integration.

3.2 System Workflow

The general system workflow begins when a user registers or logs into the application. After authentication, the user enters a tab-based interface where productivity actions can be performed through the task, habit, focus, study session, profile, and analytics modules. User actions update Firestore in real time, while gamification and statistics services continuously calculate XP, streaks, focus totals, and growth progression. These updates are reflected across the user interface through SwiftUI's reactive state management.

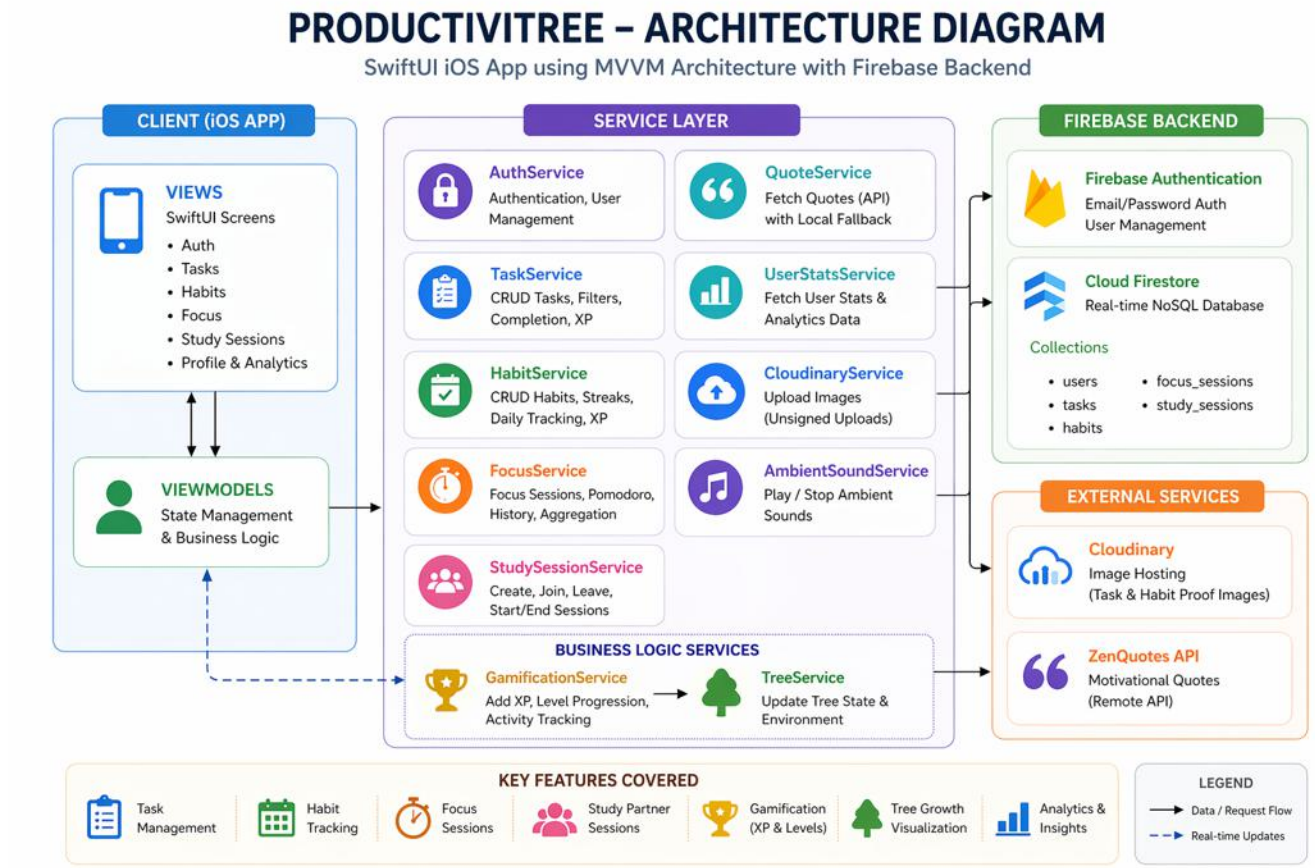
[Insert Screenshot: Overall system workflow / app entry and tab navigation here]

4. System Architecture

The application follows a layered architecture aligned with MVVM. SwiftUI views handle presentation, view models manage UI state and business coordination, and services encapsulate external integrations and database logic. Firebase provides backend authentication and persistent storage, while supporting APIs extend functionality where needed.

4.1 High-Level System Architecture Diagram

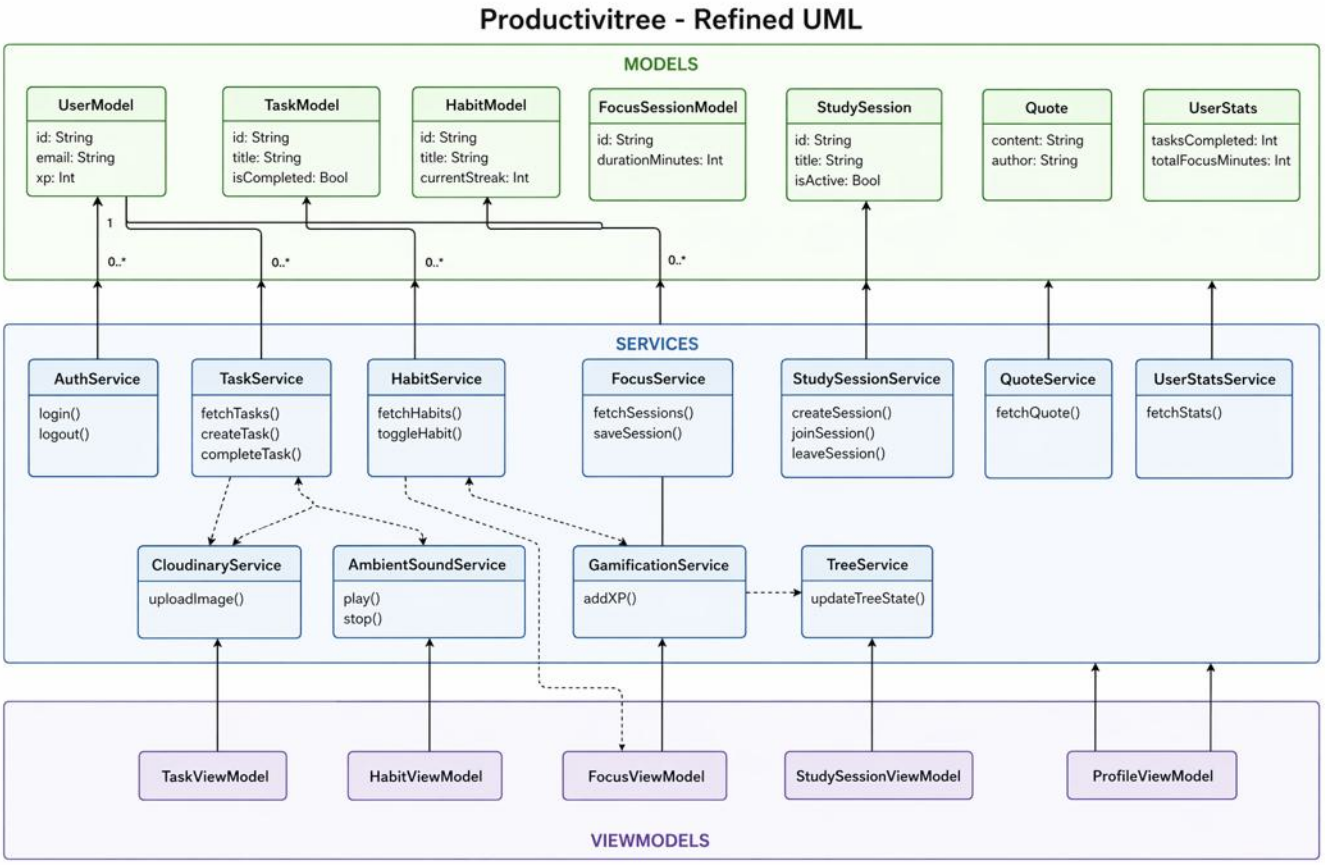
Figure 1 shows the overall system architecture of Productivitree.



The architecture begins with the user interacting through the SwiftUI interface. Those interactions are processed by the application screens and corresponding view models. Service classes perform business operations such as CRUD handling, session synchronization, image upload, quote fetching, and XP updates. Firestore and Firebase Authentication act as the core backend, while external API services support selected features such as motivational quotes and image hosting.

4.2 UML Class Diagram

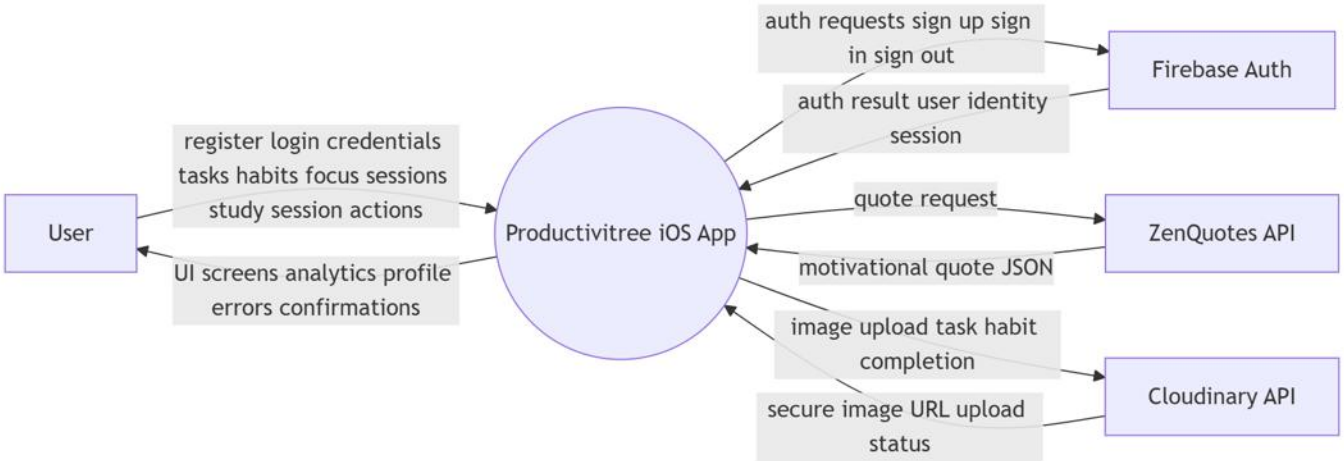
Figure 2 presents the UML class diagram of the main classes used in the system.



The class structure includes models such as UserModel, TaskModel, HabitModel, FocusSessionModel, StudySession, and UserStats. Service classes such as AuthService, TaskService, HabitService, FocusService, StudySessionService, GamificationService, TreeService, and UserStatsService are responsible for application logic and backend communication. View models coordinate these services with SwiftUI views.

4.3 DFD Level 0 (Context Diagram)

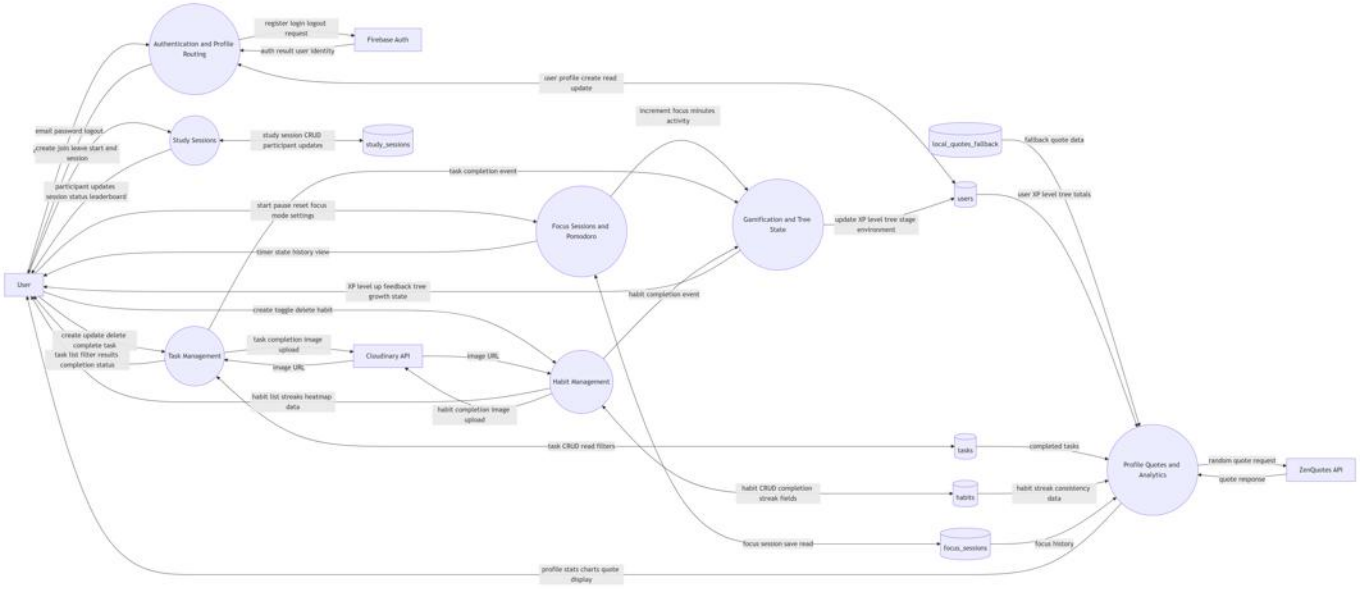
Figure 3 illustrates the context-level data flow of the system.



At the context level, the user interacts with Productivitree to manage productivity activities. The application exchanges authentication and data-storage information with Firebase services, while optional external integrations provide quote and image-upload support.

4.4 DFD Level 1

Figure 4 breaks the internal data flow into major functional processes.



The Level 1 DFD expands the application into its major modules such as authentication, task management, habit management, focus sessions, study sessions, analytics, and profile management. Each process communicates with the relevant Firestore collections and supporting services to maintain application state and user progress.

4.5 Architectural Layers

4.5.1 Presentation Layer

The presentation layer is implemented with SwiftUI views organized by feature-based modules such as Auth, Tasks, Habits, Focus, StudySessions, Profile, Analytics, and reusable UI components. This organization keeps the interface modular and maintainable.

4.5.2 ViewModel Layer

View models such as TaskViewModel, HabitViewModel, FocusViewModel, StudySessionViewModel, and ProfileViewModel manage UI state, validation, filtering, asynchronous updates, and interaction handling. This layer isolates interface logic from backend operations.

4.5.3 Service Layer

The service layer includes classes such as AuthService, TaskService, HabitService, FocusService, StudySessionService, CloudinaryService, QuoteService, TreeService, GamificationService, and UserStatsService. These services encapsulate database communication, remote calls, and business rules.

4.5.4 Backend and Integration Layer

The backend uses Firebase Authentication for login and registration, Cloud Firestore for data persistence, and Firestore security rules for access control. Cloudinary is used for optional image uploads, while the quote system uses a remote API with local fallback support.

5. Implementation and Experimental Setup

The project was implemented in Xcode using Swift, SwiftUI, Combine, Firebase SDKs, and Swift Charts. The app is organized into models, services, view models, views, resources, and Firebase configuration files. Development and testing were performed through simulator-based execution with multiple accounts to validate authentication, CRUD features, and real-time study session behavior.

The major configured Firestore collections are:

- users
- tasks
- habits
- focus_sessions
- study_sessions

The project also includes Firebase rules, image-upload support, and local resources such as quote fallback data.

[Insert Screenshot: Development environment / project structure / Xcode setup here]

6. System Functionalities and Implementation

This section presents the main modules of Productivitree and how they were implemented within the application.

6.1 Authentication

The authentication module supports email and password based registration and login. It maintains user sessions and creates corresponding user-profile records in Firestore. Auth-state monitoring allows the app to route the user to the correct interface automatically.

Main implemented features:

- User registration
- User login
- Session persistence
- Automatic auth-based screen routing
- Firestore user document creation

[Insert Screenshot: Login screen here]

[Insert Screenshot: Registration screen here]

6.2 Task Management

The task module supports full CRUD functionality. Users can create tasks with title, description, priority, and optional deadlines. The interface includes filtering options such as all tasks, today, completed, high priority, and unfinished tasks. Completing a task awards XP and can optionally include image evidence.

Main implemented features:

- Task create, read, update, and delete operations
- Priority handling: High, Medium, Low
- Deadline support
- Completion tracking
- Optional image-proof upload
- Task filtering and progress-related rewards

[Insert Screenshot: Task list screen here]

[Insert Screenshot: Add/Edit task screen here]

6.3 Habit Tracking

The habit module allows users to define daily habits and maintain streak-based progress. Completion history is recorded to support streak calculation, heatmap views, and best-streak tracking. Habit completion also supports optional image uploads and XP reward protection against duplicate awards on the same date.

Main implemented features:

- Habit CRUD operations
- Daily completion toggle
- Current streak tracking
- Best streak tracking
- Heatmap visualization
- Optional habit evidence upload

[Insert Screenshot: Habit list screen here]

[Insert Screenshot: Habit heatmap / streak view here]

6.4 Focus Sessions

The focus module provides configurable timers for productivity sessions with different focus modes. It supports focus and break durations, Pomodoro cycles, history storage, and ambient sounds. Focus time is aggregated and later reflected in profile and analytics summaries.

Main implemented features:

- Configurable focus timer
- Focus modes such as Deep Work, Learning, and Creating
- Pomodoro cycle support
- Focus history persistence
- Total focus-minute aggregation
- Ambient sound support

[Insert Screenshot: Focus timer screen here]

[Insert Screenshot: Focus history or ambient sound screen here]

6.5 Study Sessions

The study session module introduces collaborative accountability to the platform. Users can create, join, leave, and manage sessions. Session creators can start, end, or delete sessions, and Firestore listeners help keep participant data synchronized in real time.

Main implemented features:

- Study session creation
- Join and leave session
- Session detail view
- Creator-only controls
- Real-time participant updates
- Activity and leaderboard tracking

[Insert Screenshot: Study session list screen here]

[Insert Screenshot: Session details / focus together screen here]

6.6 Gamification and Tree World

Gamification is one of the most distinctive parts of Productivitree. Productive actions such as completing tasks, maintaining habits, and staying focused increase user XP. XP contributes to level progression and affects visual tree growth stages. Tree-world states also reflect activity and streak conditions to create a more engaging and rewarding interface.

Main implemented features:

- XP and level progression
- Level-up notifications
- Tree stages from seed to forest
- Environment-state logic based on activity and streaks
- Progress-based motivation and user engagement

[Insert Screenshot: Tree world / XP progress screen here]

6.7 Profile and Analytics

The profile and analytics module provides performance feedback to users through stat cards, XP progress bars, focus metrics, task completion totals, best streak values, and weekly charts. A motivational quote service further improves user engagement and reflection.

Main implemented features:

- User profile summary
- XP progress display
- Task and focus statistics
- Best-streak tracking
- Weekly analytics charts
- Motivational quote integration

[Insert Screenshot: Profile screen here]

[Insert Screenshot: Analytics screen here]

7. Data Model and Storage Design

The application uses Firestore collections to organize persistent data. Each authenticated user has an associated profile and related records. Tasks, habits, focus sessions, and study sessions are stored in structured collections to support retrieval, filtering, synchronization, and analytics.

Main data entities used in the project include:

- UserModel
- TaskModel
- HabitModel
- FocusSessionModel
- StudySession
- UserStats

The storage design ensures that user-specific data remains organized and accessible across application modules.

8. Testing, Validation, and Results

The implemented features were validated using simulator-based testing and multi-account workflow checks. Authentication was verified using multiple user sessions. CRUD functionality for tasks and habits was tested for normal flows and common edge cases. Focus sessions were tested for timer progression and stored history. Study sessions were validated for create, join, leave, and real-time participant synchronization. Analytics and profile metrics were verified against completed tasks, focus records, and streak changes.

Tested areas include:

- Registration, login, and logout
- Task CRUD and completion logic
- Habit CRUD and daily toggle behavior
- Streak calculation and heatmap updates
- Focus timer progression and history saving
- Study session real-time synchronization
- XP, level, and profile-stat updates
- Quote retrieval and fallback behavior

[Insert Screenshot: Testing outcomes / validation summary table here]

9. Work Distribution

The project work was distributed between the two group members as follows:

Rysul Aman Nirob (Roll: 2107092)

- Implemented the task management module, including task creation, listing, filtering, updating, deletion, deadlines, and completion tracking.
- Developed the focus-session features, including timer logic, configurable durations, modes, and Pomodoro behavior.
- Implemented the gamification logic, including XP rewards, level progression, and activity-based motivation.
- Built the authentication system with registration, login, session persistence, and auth-state handling.
- Worked on analytics-related functionality and user-progress insights.
- Integrated API calls such as quote retrieval and related service communication.
- Configured and integrated Firebase services for authentication and database-backed operations.
- Contributed to system architecture planning and backend-service integration details.
- Supported data-flow design and service coordination between modules.

Biposhyan Chakma (Roll: 2107128)

- Implemented the habit-tracking module, including habit CRUD, streak tracking, and habit completion logic.
- Developed the study-session module, including collaborative session creation, joining, leaving, and participant management.
- Led the frontend implementation of SwiftUI screens, layout organization, and feature-based view structure.
- Implemented CRUD flows across multiple interface modules.
- Worked on tree-world management, including tree-stage and environment-state presentation.
- Contributed to profile-related views and user-facing presentation components.
- Assisted with Firestore-connected model integration and feature-level state updates.
- Contributed to project documentation alignment with implemented modules.
- Supported testing and UI refinement across the integrated modules.

10. Challenges Faced

During implementation, several challenges were encountered:

- Managing multiple feature modules while keeping the SwiftUI state flow consistent.
- Designing a clean MVVM structure that remained flexible as new features were added.
- Handling real-time updates for collaborative study sessions.
- Preventing duplicate XP rewards for repeated habit completion on the same date.
- Coordinating cloud-based storage, authentication, and optional external integrations.
- Maintaining a balance between usability, gamification, and implementation complexity.

11. Discussion

Productivitree demonstrates that a mobile-first architecture can effectively combine personal productivity, social accountability, and gamified engagement in one application. The project benefits from modular design, where individual features such as tasks, habits, focus tracking, and analytics remain connected through shared progress data and centralized services. The use of SwiftUI and MVVM improved maintainability and allowed reactive updates across the application. Real-time study sessions and tree-world growth make the system more engaging than a conventional productivity tracker.

12. Limitations and Future Work

Although the project is functionally complete in its core areas, some limitations remain:

- Large-scale performance testing was not conducted.
- Physical-device testing was limited compared to simulator-based testing.
- Advanced notification support is not yet included.
- Offline-first synchronization is not fully implemented.
- Automated end-to-end testing can be expanded further.

Possible future improvements include:

- Push and local notification support
- Enhanced analytics and progress forecasting
- Accessibility improvements
- Better loading and error-state handling
- Richer tree-world customization
- Stronger automated testing and QA coverage

13. Conclusion

Productivitree successfully delivers a cloud-backed iOS application that integrates task management, habit tracking, focus sessions, collaborative study sessions, gamification, and analytics in a unified platform. The project applies modern mobile development practices through SwiftUI, MVVM, Firebase, and service-based modular design. It provides a practical solution for students and users who want a more engaging and socially accountable productivity experience.

14. References

[1] Apple Developer Documentation, Swift and SwiftUI.
[2] Firebase Documentation, Authentication and Cloud Firestore.
[3] Cloudinary Documentation, Image Upload API.
[4] ZenQuotes API Documentation.
[5] Course materials and related references from Mobile Computing Laboratory.